MOBILE CASHBOX

Mobile Cashbox Android SDK

Руководство по интеграции

Версия API	1.0-SNAPSHOT
Платформа	Android API 26+
Актуально	7 октября 2026

Авторизация кассира · Оплата · Возврат · Результат операции

Содержание

1. Назначение SDK 4

2. Требования и подключение 5

2.1. Системные требования . . . 5

2.2. Подключение зависимости . . . 5

2.3. Создание API . . . 5

3. Интеграция с Jetpack Compose 6

4. Интеграция с Activity или Fragment 7

5. Авторизация кассира 8

5.1. Передача учётных данных . . . 8

5.2. ConnectionState . . . 8

5.3. Наблюдение состояния в Compose . . . 8

6. Проведение оплаты 9

7. Возврат, отправка чека и логов 10

7.1. Проведение возврата . . . 10

7.2. Отправка чека . . . 10

7.3. Отправка логов . . . 10

8. Получение результата операции 11

8.1. Обработка OperationResult . . . 11

8.2. Варианты результата . . . 11

9. MobileCashboxTransaction 12

9.1. MobileCashboxMerchant . . . 12

10. Ошибки авторизации 13

11. Технические ошибки операции 14

12. Callback API 15

13. Справочник MobileCashboxApi 16

mobileCashboxApi . . . 16

connectionState . . . 16

events . . . 16

setUser . . . 16

auth . . . 16
performPayment . . . 17
performRefund . . . 17
sendReceipt . . . 17
sendLogs . . . 17
addListener . . . 17
removeListener . . . 18
close . . . 18

14. Справочник Android-интеграции 19

MobileCashboxExternalAppEffect . . . 19
MobileCashboxAndroidBridge.register . . . 19
MobileCashboxAndroidBridge.dispose . . . 19

15. Полный минимальный пример 20

16. Checklist интеграции 21

1. Назначение SDK

Mobile Cashbox SDK предоставляет Android-приложению публичный API для:

- авторизации кассира;
- запуска оплаты через поддерживаемое SoftPOS-приложение;
- запуска возврата;
- отправки чека по email;
- отправки логов Mobile Cashbox;
- получения результата операции;
- получения доступной информации об успешной транзакции.

Основная точка входа - функция `mobileCashboxApi(token)`, возвращающая `MobileCashboxApi`.

Типичный порядок работы:

1. Создать `MobileCashboxApi`.
2. Подключить Compose effect или Android bridge.
3. Передать данные кассира через `setUser()`.
4. Вызвать `auth()` и дождаться `ConnectionState.Authenticated`.
5. Запустить оплату, возврат, отправку чека или логов.
6. Получить `MobileCashboxEvent.OperationResult`.
7. Закрыть API через `close()`, когда он больше не нужен.

2. Требования и подключение

2.1. Системные требования

Параметр	Значение
Платформа	Android
Минимальная версия Android	API 26
Координаты SDK	`ru.raiffeisen:mobile-cashbox:1.0-SNAPSHOT`
Публичные асинхронные типы	`StateFlow`, `SharedFlow`
Денежный тип	`java.math.BigDecimal`
Тип даты транзакции	`java.time.LocalDateTime`

Для проведения операции на устройстве должно быть установлено поддерживаемое SoftPOS-приложение (Райффайзен Мобильный Эквайринг).

2.2. Подключение зависимости

Добавьте Maven-репозиторий, в котором опубликован SDK. Для локальной интеграции используется `mavenLocal()`:

```
// settings.gradle.kts
dependencyResolutionManagement {
    repositories {
        mavenLocal()
        google()
        mavenCentral()
    }
}
```

Добавьте зависимость в Android-приложение:

```
// app/build.gradle.kts
dependencies {
    implementation("ru.raiffeisen:mobile-cashbox:1.0-SNAPSHOT")
}
```

Если приложение использует Compose bridge, добавьте Activity Compose:

```
dependencies {
    implementation("androidx.activity:activity-compose:1.13.0")
}
```

2.3. Создание API

```
private val mobileCashbox = mobileCashboxApi(BuildConfig.MOBILE_CASHBOX_TOKEN)
```

Не храните рабочий токен в исходном коде или системе контроля версий.

3. Интеграция с Jetpack Compose

Для Compose используйте `MobileCashboxExternalAppEffect`.

Effect должен вызываться безусловно у корня Compose-дерева. Это гарантирует своевременную регистрацию Android Activity Result launcher.

```
class MainActivity : ComponentActivity() {
    private val mobileCashbox =
        mobileCashboxApi(BuildConfig.MOBILE_CASHBOX_TOKEN)

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MobileCashboxExternalAppEffect(mobileCashbox)
            AppContent(mobileCashbox)
        }
    }

    override fun onDestroy() {
        mobileCashbox.close()
        super.onDestroy()
    }
}
```

Не помещайте `MobileCashboxExternalAppEffect` внутрь условно отображаемого экрана, который может впервые появиться после запуска Activity.

4. Интеграция с Activity или Fragment

Для View-based интерфейса используйте `MobileCashboxAndroidBridge`.

Bridge необходимо зарегистрировать до перехода Activity или Fragment в состояние `STARTED`.

```
class PaymentActivity : ComponentActivity() {
    private val mobileCashbox =
        mobileCashboxApi(BuildConfig.MOBILE_CASHBOX_TOKEN)

    private val mobileCashboxBridge =
        MobileCashboxAndroidBridge.register(this, mobileCashbox)

    override fun onDestroy() {
        mobileCashboxBridge.dispose()
        mobileCashbox.close()
        super.onDestroy()
    }
}
```

Для View-based интеграции при завершении жизненного цикла необходимо вызвать оба метода:

- `MobileCashboxAndroidBridge.dispose();`
- `MobileCashboxApi.close()`.

5. Авторизация кассира

5.1. Передача учётных данных

SDK поддерживает авторизацию по email, телефону или обоим значениям.

```
mobileCashbox.setUser(
    AuthorizationCredential.Email("cashier@example.com")
)

mobileCashbox.setUser(
    AuthorizationCredential.Phone("79991234567")
)

mobileCashbox.setUser(
    AuthorizationCredential.EmailAndPhone(
        email = "cashier@example.com",
        phone = "79991234567",
    )
)
```

После передачи учётных данных вызовите:

```
mobileCashbox.auth()
```

`auth()` запускает операцию асинхронно. Текущее состояние доступно в `connectionState`.

5.2. ConnectionState

Состояние	Описание
`Idle`	Начальное состояние. Авторизация ещё не запускалась.
`Authenticating`	Выполняется авторизация кассира.
`Authenticated`	Авторизация завершена успешно. Можно запускать оплату или возврат.
`AuthenticationError(reason)`	Авторизация завершена ошибкой. Причина находится в `reason`.

5.3. Наблюдение состояния в Compose

```
@Composable
fun AuthorizationState(mobileCashbox: MobileCashboxApi) {
    val state by mobileCashbox.connectionState.collectAsState()

    when (state) {
        ConnectionState.Idle -> ShowAuthorizationForm()
        ConnectionState.Authenticating -> ShowProgress()
        ConnectionState.Authenticated -> ShowOperations()
        is ConnectionState.AuthenticationError -> {
            ShowAuthorizationError(state.reason)
        }
    }
}
```

6. Проведение оплаты

Запускайте оплату после получения `ConnectionState.Authenticated`.

```
mobileCashbox.performPayment(
    PaymentData(
        amount = BigDecimal("123.45"),
    )
)
```

`performPayment()` возвращает `Unit` и запускает операцию асинхронно. Итог приходит через `MobileCashboxEvent.OperationResult` с типом `PAYMENT`.

Перед вызовом рекомендуется проверить, что сумма является положительной.

Не запускайте следующую операцию до получения результата текущей операции.

7. Возврат, отправка чека и логов

7.1. Проведение возврата

Для возврата передайте сумму, идентификатор и дату исходной транзакции.

```
mobileCashbox.performRefund(
    RefundData(
        amount = BigDecimal("123.45"),
        transactionId = "source-transaction-id",
        transactionDate = LocalDateTime.parse(
            "2026-09-28T14:47:56"
        ),
    )
)
```

Поле	Описание
`amount`	Сумма возврата.
`transactionId`	Идентификатор исходной транзакции, предусмотренный договорённостью интеграции.
`transactionDate`	Дата и время исходной транзакции.

`performRefund()` возвращает `Unit` и запускает операцию асинхронно. Итог приходит через `MobileCashboxEvent.OperationResult` с типом `REFUND`.

7.2. Отправка чека

Для отправки чека передайте email получателя, идентификатор и дату исходной операции оплаты.

```
mobileCashbox.sendReceipt(
    ReceiptData(
        receiptEmail = "customer@example.com",
        transactionId = "source-transaction-id",
        transactionDate = LocalDateTime.parse(
            "2026-09-29T14:30:00"
        ),
    )
)
```

Поле	Описание
`receiptEmail`	Валидный непустой email получателя чека.
`transactionId`	Идентификатор исходной операции оплаты.
`transactionDate`	Дата и время исходной операции оплаты в локальном времени.

`sendReceipt()` возвращает `Unit` и запускает операцию асинхронно. Итог приходит через `MobileCashboxEvent.OperationResult` с типом `RECEIPT`.

7.3. Отправка логов

```
mobileCashbox.sendLogs()
```

`sendLogs()` использует ранее переданные данные кассира, возвращает `Unit` и запускает операцию асинхронно. Итог приходит через `MobileCashboxEvent.OperationResult` с типом `LOGS`.

8. Получение результата операции

`events` - поток одноразовых событий. Начните его сбор до запуска любой операции.

```
LaunchedEffect(mobileCashbox) {
    mobileCashbox.events.collect { event ->
        when (event) {
            is MobileCashboxEvent.OperationResult -> {
                handleOperationResult(event)
            }
        }
    }
}
```

8.1. Обработка OperationResult

```
private fun handleOperationResult(
    event: MobileCashboxEvent.OperationResult,
) {
    when (val result = event.result) {
        is MobileCashboxOperationResult.Success -> {
            showSuccess(
                operationType = event.operationType,
                transaction = result.transaction,
            )
        }

        is MobileCashboxOperationResult.Failure -> {
            showFailure(
                code = result.code,
                description = result.description,
            )
        }

        is MobileCashboxOperationResult.TechnicalFailure -> {
            showTechnicalFailure(result.reason)
        }
    }
}
```

8.2. Варианты результата

Результат	Описание
`Success(transaction)`	Операция завершена успешно. `transaction` может отсутствовать.
`Failure(code, description)`	Операция отклонена. Код и описание могут отсутствовать.
`TechnicalFailure(reason)`	Операция не завершена из-за технической причины.

Успешность уже определена SDK. Пользователю API не требуется дополнительно проверять `transaction.status`.

9. MobileCashboxTransaction

`MobileCashboxOperationResult.Success` содержит nullable-свойство `transaction`.

Все поля `MobileCashboxTransaction` также nullable: конкретный результат может содержать только часть данных.

Поле	Тип	Описание
`terminalId`	`Int?`	Идентификатор терминала.
`operationDay`	`Int?`	Операционный день.
`transactionNumber`	`Int?`	Номер транзакции.
`reversible`	`Boolean?`	Признак возможности обратной операции.
`externalTerminalId`	`String?`	Внешний идентификатор терминала.
`dateTime`	`String?`	Дата и время операции в формате источника.
`operationExternalId`	`String?`	Внешний идентификатор операции.
`status`	`String?`	Информационный статус транзакции.
`statusDescription`	`String?`	Описание статуса.
`instrument`	`String?`	Платёжный инструмент.
`type`	`String?`	Тип транзакции.
`paymentSystemType`	`String?`	Платёжная система.
`amount`	`BigDecimal?`	Сумма операции.
`currencyCode`	`String?`	Код валюты.
`companyName`	`String?`	Название компании.
`merchant`	`MobileCashboxMerchant?`	Информация о торговой точке.

9.1. MobileCashboxMerchant

Поле	Тип	Описание
`id`	`String?`	Идентификатор торговой точки.
`name`	`String?`	Название торговой точки.
`address`	`String?`	Адрес торговой точки.
`phoneNumber`	`String?`	Телефон торговой точки.

10. Ошибки авторизации

`ConnectionState.AuthenticationError` содержит `ConnectionFailureReason`.

Причина	Описание
`InvalidToken`	Токен не принят.
`InvalidCredential`	Учётные данные отсутствуют или некорректны.
`CashierNotRegistered`	Кассир не зарегистрирован.
`NoPermissionForMerchant`	У кассира нет разрешения для торговой точки.
`CashierNotActivated`	Кассир не активирован.
`NetworkUnavailable`	Сеть недоступна.
`Timeout`	Превышено время ожидания.
`ServerError(statusCode)`	Сервис вернул серверную ошибку.
`BadRequest(problemCode)`	Запрос отклонён как некорректный.
`Forbidden(problemCode)`	Доступ запрещён.
`Unknown(message)`	Неизвестная ошибка.

11. Технические ошибки операции

`MobileCashboxOperationResult.TechnicalFailure` содержит `MobileCashboxTechnicalFailureReason`.

Причина	Описание
`RESULT_DATA_KEY_MISSING`	Результат операции не содержит ожидаемых данных.
`RESULT_DATA_BLANK`	Получены пустые данные результата.
`INVALID_RESULT_JSON`	Формат результата не поддерживается.
`PLATFORM_CANCELED`	Операция отменена на уровне Android.
`APPLICATION_NOT_INSTALLED`	Поддерживаемое SoftPOS-приложение не найдено.
`INITIALIZATION_FAILED`	Операцию не удалось подготовить к запуску.

12. Callback API

Вместо Flow приложение может зарегистрировать `MobileCashboxListener`.

```
private val listener = object : MobileCashboxListener {
    override fun onStateChanged(state: ConnectionState) {
        renderConnectionState(state)
    }

    override fun onEvent(event: MobileCashboxEvent) {
        if (event is MobileCashboxEvent.OperationResult) {
            handleOperationResult(event)
        }
    }
}
```

Регистрация и удаление listener:

```
mobileCashbox.addListener(listener)

// Когда listener больше не нужен:
mobileCashbox.removeListener(listener)
```

Callback можно использовать для обновления Android UI.

13. Справочник MobileCashboxApi

mobileCashboxApi

Параметр	Значение
Сигнатура	`MobileCashboxApi mobileCashboxApi(String token)`
Входные параметры	`token` - токен доступа.
Возвращаемое значение	Экземпляр `MobileCashboxApi`.
Описание	Создаёт API для выполнения операций Mobile Cashbox.

connectionState

Параметр	Значение
Тип	`StateFlow<ConnectionState>`
Описание	Текущее состояние авторизации.

events

Параметр	Значение
Тип	`SharedFlow<MobileCashboxEvent>`
Описание	Одноразовые события, включая результаты оплаты, возврата, отправки чека и логов.

setUser

Параметр	Значение
Сигнатура	`void setUser(AuthorizationCredential credential)`
Входные параметры	Учётные данные кассира.
Возвращаемое значение	Нет.
Описание	Устанавливает данные кассира для последующей авторизации и операций.

auth

Параметр	Значение
Сигнатура	`void auth()`
Входные параметры	Нет.
Возвращаемое значение	Нет.
Описание	Асинхронно запускает авторизацию. Результат отражается в `connectionState`.

performPayment

Параметр	Значение
Сигнатура	`void performPayment(PaymentData paymentData)`
Входные параметры	Данные оплаты.
Возвращаемое значение	Нет.
Описание	Запускает оплату. Итог приходит через `OperationResult`.

performRefund

Параметр	Значение
Сигнатура	`void performRefund(RefundData refundData)`
Входные параметры	Данные возврата.
Возвращаемое значение	Нет.
Описание	Запускает возврат. Итог приходит через `OperationResult`.

sendReceipt

Параметр	Значение
Сигнатура	`void sendReceipt(ReceiptData receiptData)`
Входные параметры	Email получателя и ссылка на исходную операцию оплаты.
Возвращаемое значение	Нет.
Описание	Запускает отправку чека. Итог приходит через `OperationResult` с типом `RECEIPT`.

sendLogs

Параметр	Значение
Сигнатура	`void sendLogs()`
Входные параметры	Нет. Используются ранее переданные данные кассира.
Возвращаемое значение	Нет.
Описание	Запускает отправку логов. Итог приходит через `OperationResult` с типом `LOGS`.

addListener

Параметр	Значение
Сигнатура	`void addListener(MobileCashboxListener listener)`
Входные параметры	Listener состояния и событий.
Возвращаемое значение	Нет.
Описание	Регистрирует callback listener.

removeListener

Параметр	Значение
Сигнатура	`void removeListener(MobileCashboxListener listener)`
Входные параметры	Ранее зарегистрированный listener.
Возвращаемое значение	Нет.
Описание	Удаляет callback listener.

close

Параметр	Значение
Сигнатура	`void close()`
Входные параметры	Нет.
Возвращаемое значение	Нет.
Описание	Освобождает ресурсы API. После вызова экземпляр больше не используется.

14. Справочник Android-интеграции

MobileCashboxExternalAppEffect

Параметр	Значение
Сигнатура	`@Composable void MobileCashboxExternalAppEffect(MobileCashboxApi lib)`
Входные параметры	Экземпляр `MobileCashboxApi`.
Возвращаемое значение	Нет.
Описание	Подключает запуск внешнего приложения к Compose UI.

MobileCashboxAndroidBridge.register

Параметр	Значение
Сигнатура	`MobileCashboxAndroidBridge register(ActivityResultCaller caller, MobileCashboxApi lib)`
Входные параметры	Activity/Fragment и экземпляр API.
Возвращаемое значение	Зарегистрированный bridge.
Описание	Подключает запуск внешнего приложения к Activity или Fragment.

MobileCashboxAndroidBridge.dispose

Параметр	Значение
Сигнатура	`void dispose()`
Входные параметры	Нет.
Возвращаемое значение	Нет.
Описание	Освобождает ресурсы bridge.

15. Полный минимальный пример

```
class CashboxActivity : ComponentActivity() {
    private val mobileCashbox =
        mobileCashboxApi(BuildConfig.MOBILE_CASHBOX_TOKEN)

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            MobileCashboxExternalAppEffect(mobileCashbox)

            LaunchedEffect(mobileCashbox) {
                mobileCashbox.events.collect { event ->
                    if (event is MobileCashboxEvent.OperationResult) {
                        handleOperationResult(event)
                    }
                }
            }

            CashboxScreen(
                connectionState = mobileCashbox.connectionState
                    .collectAsState()
                    .value,
                onAuthorize = { email, phone ->
                    mobileCashbox.setUser(
                        AuthorizationCredential.EmailAndPhone(
                            email = email,
                            phone = phone,
                        )
                    )
                    mobileCashbox.auth()
                },
                onPayment = { amount ->
                    mobileCashbox.performPayment(
                        PaymentData(amount)
                    )
                },
            )
        }
    }

    override fun onDestroy() {
        mobileCashbox.close()
        super.onDestroy()
    }
}
```

16. Checklist интеграции

1. SDK добавлен в зависимости Android-приложения.
2. Токен передаётся без размещения в исходном коде.
3. Compose effect или Android bridge подключён до запуска операций.
4. Приложение наблюдает `connectionState`.
5. Все операции доступны только после `Authenticated`.
6. Сбор `events` начинается до запуска операции.
7. Одновременно выполняется не более одной операции.
8. Обрабатываются `Success`, `Failure` и `TechnicalFailure`.
9. `Success.transaction` обрабатывается как nullable.
10. При завершении lifecycle вызываются `close()` и, при необходимости, `dispose()`.